

REST · JSON · JWT

Frontend API Design

Auth, CV Parsing, Matching Engine & Interview Prep

Base URL `http://localhost:8080/api/v1`
Auth `Bearer JWT (except public endpoints)`

Contents

01 Authentication Endpoints

Register, login, refresh, logout, current user

02 CV Parser Endpoints

Upload, status, edit, export, suggestions

03 Matching Engine Endpoints

Job matches and match detail analysis

04 Interview Prep Endpoints

Practice sessions, answers, feedback, history

05 Dashboard Endpoint

Aggregated account overview

06 Error Responses

Standard error envelope & codes

07 Rate Limits

Per-endpoint throttling

08 Notes

Implementation notes for frontend integration

01 · Authentication Endpoints

JWT-based auth. Access tokens are short-lived; refresh tokens renew a session.

POST /auth/register

Create a new user account.

REQUEST BODY

```
{
  "email": "user@example.com",
  "password": "securepassword123",
  "name": "John Doe"
}
```

RESPONSE 201

```
{
  "user": {
    "id": "uuid",
    "email": "user@example.com",
    "name": "John Doe",
    "role": "user"
  },
  "access": "jwt_access_token",
  "refresh": "jwt_refresh_token"
}
```

POST /auth/login

Authenticate with email and password.

REQUEST BODY

```
{
  "email": "user@example.com",
  "password": "securepassword123"
}
```

RESPONSE 200

```
{
  "access": "jwt_access_token",
  "refresh": "jwt_refresh_token",
  "user": {
    "id": "uuid",
    "email": "user@example.com",
    "name": "John Doe",
    "role": "user"
  }
}
```

POST /auth/refresh

Exchange a refresh token for a new token pair.

REQUEST BODY

```
{
  "refresh": "jwt_refresh_token"
}
```

RESPONSE 200

```
{
  "access": "new_jwt_access_token",
  "refresh": "new_jwt_refresh_token"
}
```

POST /auth/logout

Invalidate the given refresh token.

REQUEST BODY

```
{
  "refresh": "jwt_refresh_token"
}
```

RESPONSE 200

```
{
  "message": "Successfully logged out"
}
```

GET**/auth/me**

Return the currently authenticated user.

RESPONSE **200**

```
{
  "user": {
    "id": "uuid",
    "email": "user@example.com",
    "name": "John Doe",
    "role": "user",
    "created_at": "2024-01-01T00:00:00Z"
  }
}
```

02 · CV Parser Endpoints

Upload, inspect, and refine a parsed CV, then export the result.

POST /cv/upload

Upload a CV file for asynchronous parsing.

REQUEST BODY

```
file: <PDF file> (multipart/form-data)
```

RESPONSE 202

```
{
  "cv_id": "uuid",
  "status": "processing",
  "message": "CV uploaded and is being processed"
}
```

GET /cv/{cv_id}/status

Poll the parsing status and retrieve parsed data once complete.

RESPONSE 200

```
{
  "cv_id": "uuid",
  "status": "completed",
  "parsed_data": {
    "name": "John Doe",
    "email": "john@example.com",
    "phone": "+1234567890",
    "skills": ["Python", "Django", "React"],
    "experience": [...],
    "education": [...]
  },
  "optimization_suggestions": [
    {
      "section": "skills",
      "suggestion": "Add more technical skills",
      "priority": "high"
    }
  ]
}
```

PATCH /cv/{cv_id}

Edit a single field of the parsed CV data.

REQUEST BODY

```
{
  "field": "email",
  "value": "newemail@example.com"
}
```

RESPONSE 200

```
{
  "cv_id": "uuid",
  "field": "email",
  "updated_value": "newemail@example.com"
}
```

GET /cv/{cv_id}/export

Export the CV as a binary PDF file.

RESPONSE 200

```
Content-Type: application/pdf
Binary PDF file
```

POST /cv/{cv_id}/suggestions/accept

Accept and apply an optimization suggestion.

REQUEST BODY

```
{
  "suggestion_id": "uuid"
}
```

RESPONSE 200

```
{
  "message": "Suggestion accepted and applied"
}
```

03 · Matching Engine Endpoints

Score a parsed CV against open roles and inspect individual match detail.

GET `/matches/{cv_id}`

List ranked job matches for a CV.

QUERY PARAMETERS

<code>min_score</code>	Minimum match score, 0–1 (optional)
<code>limit</code>	Number of results, default 20 (optional)
<code>offset</code>	Pagination offset, default 0 (optional)
<code>sort</code>	Sort by score or date, default score (optional)

RESPONSE 200

```
{
  "cv_id": "uuid",
  "total_matches": 150,
  "matches": [
    {
      "job_id": "uuid",
      "job_title": "Senior Python Developer",
      "company": "Tech Corp",
      "match_score": 0.92,
      "matched_skills": ["Python", "Django", "PostgreSQL"],
      "missing_skills": ["AWS", "Docker"],
      "location": "Remote",
      "salary_range": "$100k-$150k"
    }
  ]
}
```

GET `/matches/{cv_id}/{job_id}`

Return a detailed strengths/gaps analysis for one match.

RESPONSE 200

```
{
  "cv_id": "uuid",
  "job_id": "uuid",
  "match_score": 0.92,
  "job_details": {
    "title": "Senior Python Developer",
    "company": "Tech Corp",
    "description": "...",
    "requirements": [...],
    "location": "Remote",
    "salary": "$120k"
  },
  "analysis": {
    "strengths": ["Strong Python experience", "Relevant projects"],
    "gaps": ["Missing cloud experience"],
    "recommendations": ["Consider adding AWS certification"]
  }
}
```

04 · Interview Prep Endpoints

Practice sessions with per-answer feedback and rolling history stats.

POST /interview/session

Start a new practice interview session for a job.

REQUEST BODY

```
{
  "job_id": "uuid",
  "focus_area": "python"
}
```

RESPONSE 201

```
{
  "session_id": "uuid",
  "question": "Explain the difference between list and tuple in Python",
  "difficulty": "medium",
  "time_limit": 300
}
```

POST /interview/session/{session_id}/answer

Submit an answer for asynchronous evaluation.

REQUEST BODY

```
{
  "answer": "Lists are mutable, tuples are immutable...",
  "time_taken": 45
}
```

RESPONSE 202

```
{
  "answer_id": "uuid",
  "status": "processing",
  "message": "Answer submitted for evaluation"
}
```

GET /interview/answer/{answer_id}

Retrieve scored feedback once evaluation completes.

RESPONSE 200

```
{
  "answer_id": "uuid",
  "status": "completed",
  "feedback": {
    "score": 8.5,
    "strengths": ["Good explanation of mutability"],
    "improvements": ["Could mention memory usage"],
    "suggested_answer": "..."
  }
}
```

GET**/interview/history**

List past sessions with aggregated performance stats.

QUERY PARAMETERS

limit	Number of sessions, default 10 (optional)
offset	Pagination offset, default 0 (optional)
job_id	Filter by job ID (optional)

RESPONSE 200

```
{
  "sessions": [
    {
      "session_id": "uuid",
      "job_id": "uuid",
      "job_title": "Python Developer",
      "date": "2024-01-15T10:00:00Z",
      "average_score": 7.8,
      "questions_answered": 5
    }
  ],
  "aggregated_stats": {
    "total_sessions": 25,
    "average_score": 7.5,
    "strong_areas": ["Python basics", "OOP"],
    "weak_areas": ["Async programming"]
  }
}
```

05 · Dashboard Endpoint

A single aggregated snapshot for the home / overview screen.

GET **/dashboard**

Return CV, match, and interview summaries for the current user.

RESPONSE **200**

```
{
  "user": {
    "id": "uuid",
    "name": "John Doe",
    "email": "john@example.com"
  },
  "cv_summary": {
    "total_cvs": 3,
    "latest_cv_id": "uuid",
    "latest_cv_score": 0.85,
    "average_cv_score": 0.78
  },
  "match_summary": {
    "total_matches": 150,
    "high_priority_matches": 25,
    "average_match_score": 0.75
  },
  "interview_summary": {
    "practice_sessions": 15,
    "average_score": 7.5,
    "strong_areas": ["Python", "Django"],
    "weak_areas": ["System Design"]
  }
}
```

06 · Error Responses

Every endpoint returns errors in a single, consistent envelope.

```
{
  "error": {
    "code": "ERROR_CODE",
    "message": "Human-readable error message"
  },
  "correlation_id": "uuid-for-tracking"
}
```

COMMON ERROR CODES

AUTH_MISSING_TOKEN	Missing authorization header
AUTH_INVALID_TOKEN	Invalid or expired token
VALIDATION_ERROR	Request validation failed
RATE_LIMIT_EXCEEDED	Too many requests
SERVICE_UNAVAILABLE	Downstream service error
NOT_FOUND	Resource not found

07 · Rate Limits

Auth endpoints	5 / minute
CV upload	2 / minute
Jobs endpoints	200 / minute
Interview endpoints	10 / minute
Default	100 / minute

Rate-limit headers are included on every response: `X-RateLimit-Limit-Minute` and `X-RateLimit-Remaining-Minute`.

08 · Notes

- 1 All authenticated endpoints require an `Authorization: Bearer <token>` header.
- 2 Tokens should be refreshed via `/auth/refresh` before they expire.
- 3 Use the `correlation_id` from error responses when reporting issues.
- 4 File uploads must use `multipart/form-data`.
- 5 All datetime fields are ISO 8601, in UTC.